EXPO DESIGN AI: SYSTEM AUDIT & GAP ANALYSIS

Complete Architectural Review across Database, Ingestion, Indexing, Knowledge Hub & Q&A

System: Expo Design AI (Offline Engineering Assistant) | **Root Path:** C:\EH\ExpoDesignAI - Copy | **Scope:** Database, Pipeline, Indexing, Hub, Q&A | **Date:** October 2026

1. Executive Summary & Audit Scorecard

A comprehensive code-level audit was conducted across all operational subsystems of **Expo Design AI**. The platform is designed as an offline, zero-data-leakage engineering intelligence engine capable of analyzing architectural, structural, and MEP packages. While the foundational processing modules (PyMuPDF geometry parsing, Tesseract/Qwen2.5-VL vision fallbacks, and hybrid FAISS/BM25 retrieval) are sound, several critical structural bottlenecks, data silos, and hardcoded guards prevent new user-uploaded documents from functioning correctly.

Architectural Pillar	Audit Status	Key Findings & Primary Failure Modes
1. Database & Workflow	⚠ **Degraded**	4 fragmented fact tables with zero relational unification. Re-upload bug in `pipeline.py` (`if existing: continue`) silently skips revisions. Dual job tracking divergence between `ingestion-jobs.sqlite3` and `db.JobProgress`.
2. Upload & Processing	🔴 **Broken for New Docs**	Table layout engine extracts `header_inherited_markdown`, but `_units_from_meta()` in `main.py` omits it from vectorization. CAD/DWG pipeline bypasses structured table extraction completely.
3. AI Indexing	⚠ **Restricted**	`DocumentMemory` is hardcoded to an allowlist of exactly 2 benchmark PDFs in `config.py`. All newly uploaded documents bypass memory generation completely. FAISS in-memory dictionary lacks cross-worker cache sync.
4. Concept Wiki & Hub	🔴 **Disconnected**	`KnowledgeHub` is 100% in-memory with zero SQLite persistence (data lost on restart). `EntityExtractor` is never invoked during document ingestion. SQL tables `Requirement`, `Evidence`, and `GraphEdge` remain empty.
5. Question Answering (Q&A)	⚠ **Inconsistent**	Dual pipeline divergence: `/ask_stream` vs `/api/query_document` use completely different retrieval logic. Structured DB lookup in RAG is hardcoded via regex to TOS/TOPR and legend codes only, ignoring all other schedules.

> **Primary Takeaway:** The core extraction engines work, but the application is disconnected: extracted table headers are discarded before vectorization, newly uploaded files are blocked from AI Document Memory by a benchmark allowlist, and database records are bypassed at query time by rigid regular expressions.

2. System Architecture & Root Cause Flow

2.1 End-to-End System Processing Architecture

The diagram below maps the intake, storage, and querying paths across the application, highlighting identified gaps:

2.2 Root Cause Flow: Why Newly Uploaded Documents Fail

When an engineer uploads a new drawing or calculation sheet, three cascading bottlenecks cause incorrect or refused answers:

> **Compounding Failure Sequence: 1. Allowlist Gate:** `config.document_memory_doc_allowlist` skips memory building (0 verified facts in DB). | **2. Header Loss:** `_units_from_meta()` strips multi-level headers from vector chunks. | **3. Query Miss:** `retrieve_context()` only checks DB for TOS/TOPR, so standard schedules fall back to degraded RAG chunks.

3. In-Depth Technical Audit: Pillars 1, 2 & 3

Pillar 1: Database & Workflow Architecture

- **Four Disconnected Fact Silos:** Extracted data is fragmented across four tables with conflicting schemas:
  1. `db.StructuredRecord`: Cell-level table coordinates (`tower`, `level`, `field`, `value`).
  2. `db.ScheduleRow`: JSON row dictionaries (`table_name`, `row_key`, `row_values`).
  3. `db.Quantity`: Scalar metrics (`building`, `metric="TOS"`, `value`).
  4. `db.DocumentMemory`: Relationship-aware fact claims (`entity`, `section`, `claim`, `evidence_quote`).

  There is no foreign key, UUID, or entity registry linking these four representations. An update in one leaves all others stale.
- **Silent Re-Upload Skip Bug:** In `services/pipeline.py` (lines 305–309 & 417–424):

  ```
  existing = session.query(db.StructuredRecord).filter_by(
      project_id=project_id, doc=filename, page=i + 1, entity=row_key, field=col
  ).first()
  if existing: continue  # <-- SILENTLY SKIPS UPDATES!
  ```

  When a revised revision or corrected document is uploaded under the same filename, existing rows are skipped. **Obsolete data remains in the database permanently.**
- **Dual Job Queue Progress:** `services/jobs.py` uses `ingestion-jobs.sqlite3` with OS-level file locking, while `db.py` uses `JobProgress` in `expo.db`. If an error aborts processing, UI progress polling can desynchronize from the actual runner state.

Pillar 2: Uploading & Ingestion Pipeline

- **The Header-Inheritance Ingestion Omission:** In `main.py` (lines 2752–2763), `layout_table_engine` builds `header_inherited_markdown` and assigns it to `p["header_inherited_tables"]`. However, in `_units_from_meta()` (lines 1930–1951):

  ```
  for pg in meta["pages"]:
      body = (pg.get("text") or "")
      tbl = (pg.get("tables") or "")
      # pg.get("header_inherited_tables") is COMPLETELY IGNORED!
  ```

  Because `meta["pages"]` contains raw text, `index_structured()` produces chunks > 0, and the text fallback is never triggered. **Hierarchical table structures are completely lost from vector indexing.**
- **CAD / DWG Processing Isolation:** `cad.py` converts CAD models to raster images and visual summaries, but bypasses the table layout engine and does not emit `StructuredRecord` rows for door/window or column schedules.
- **Upload Security:** `save_upload_capped()` in `services/ingestion.py` enforces `max_upload_size_mb` (2000 MB cap) and prevents disk exhaustion attacks. Path traversal checks in `contained_upload_path()` correctly validate filenames.

Pillar 3: AI Indexing Architecture

- **Hardcoded Document Memory Allowlist:** In `config.py` (lines 74–77), `document_memory_doc_allowlist` is hardcoded to only two benchmark files. In `intelligence/document_memory.py` (line 571), `is_allowlisted(filename)` rejects all user-uploaded files, skipping `DocumentMemory` generation entirely.
- **Chunk Boundary Loss:** `RecursiveCharacterTextSplitter` in `main.py` splits text at 800-character windows. While `[TABLE ROW]` units are exempt, complex engineering specifications and design notes get fragmented, disconnecting criteria clauses from their parent requirements.
- **BM25 Cache Invalidation:** `_bm25_cache` in `main.py` (line 2013) is keyed on `id(vs)` (in-memory pointer). In a multi-worker deployment, index updates on disk are not detected by other processes, serving stale BM25 keywords.

4. Technical Audit: Pillars 4 & 5 and the Concept Wiki

Pillar 4: Concept Wiki & Knowledge Hub Disconnect

- **100% In-Memory Ephemeral Storage:** `KnowledgeHub` in `knowledge/hub.py` stores entities in `self._entities: Dict` and graph relationships in `self._graph: KnowledgeGraph`. It has zero SQLite loading or persistence methods. Every server restart clears the entire Knowledge Hub.
- **Entity Extraction Never Executed:** `knowledge/extraction.py` defines an `EntityExtractor`, but it is **never called in `main.py`** during document ingestion. Dedicated SQL tables (`db.Requirement`, `db.Evidence`, `db.GraphEdge`) remain permanently unpopulated for document uploads.

4.2 The Solution: Unified "Concept Wiki" Layer

To replace fragile ad-hoc regex matching with deterministic engineering intelligence, the platform must introduce a centralized **Concept Wiki** layer:

Core Responsibilities of the Concept Wiki:
1. **Canonical Entity Normalization:** Resolves variations across drawings (e.g. "T-1", "Tower 01", "Building 1") to a single canonical concept UUID (e.g. `TOWER_1`).
2. **Cross-Document Synthesis:** Links structural, architectural, and MEP schedules under the parent entity.
3. **Persistent Knowledge Graph:** Loads and persists nodes and edges to `db.GraphEdge` and `db.Requirement` on startup.
4. **Universal Context Assembly:** Injects all associated records into the context window without fragile regex matching.

Pillar 5: Question Answering (Q&A) & Routing

- **Dual Endpoint Divergence:**
  - `/ask_stream` (UI): Uses `exact_answer()` → `memory_lookup()` → hybrid RAG retrieval → local LLM streaming → `guarded_answer_with_repair()`.
  - `/api/query_document` (API/Forensics): Calls `structured_query.py`, restricts structured lookups to single-field GFA/NSA/NISA/BUA/GA queries, and falls back to un-guarded local LLM calls.
- **Restrictive Database Lookup Regex in RAG:** In `main.py` (lines 2261–2272), database structured tables are only queried if the query matches `(TOS|TOPR)...tower (\d+)` or a legend code like `LX-PT`. Other schedule queries never touch the database tables.
- **Disabled Verification Safeguards:** In `config.py`, `enable_self_check`, `enable_query_expansion`, and `enable_metadata_prefilter` are all set to `False` by default.

5. Prioritized Remediation Roadmap & Quick Fixes

Phase	Remediation Target	Concrete Code Action & Target File
Phase 1 (Immediate)	**1. Fix Re-Upload Stale Data** `services/pipeline.py`	Replace `if existing: continue` (lines 308, 424) with an explicit SQL `UPDATE` that refreshes `value`, `raw_value`, `confidence`, and `status` when a file revision is re-uploaded.
Phase 1 (Immediate)	**2. Wire Header Tables into Vector Store** `main.py`	In `_units_from_meta()` (line 1944), add: `inh = pg.get("header_inherited_tables") or "" if inh.strip(): units.append(("[STRUCTURED TABLE]\n" + inh, pg.get("page")))`
Phase 1 (Immediate)	**3. Remove Document Memory Allowlist** `config.py` & `document_memory.py`	In `config.py`, add `document_memory_allow_all: bool = True` and update `is_allowlisted()` in `document_memory.py` (line 571) to return `True` when enabled.
Phase 2 (Medium-Term)	**4. Unify Query Routing** `main.py` & `document_query.py`	Consolidate `/ask_stream` and `/api/query_document` into a shared `QueryRouter` service so both headless tests and web UI receive identical verified answers.
Phase 2 (Medium-Term)	**5. Generalize Schedule Lookup** `main.py`	Replace lines 2261–2272 (strict TOS/TOPR regex) with entity extraction against `db.ScheduleRow` and `db.StructuredRecord` across all engineering disciplines.
Phase 2 (Medium-Term)	**6. Persist Knowledge Hub** `knowledge/hub.py`	Connect `KnowledgeHub` to SQLite models (`Requirement`, `Evidence`, `GraphEdge`) so extracted entities persist across server restarts.
Phase 3 (Strategic)	**7. Full Concept Wiki & UI Browser** `db.py` & `ui/app.html`	Implement `Concept` and `Synonym` tables in `db.py` with an interactive graph explorer in the UI allowing engineers to inspect project elements across documents.

Immediate Code Implementation: Phase 1 Quick Fixes

```
# Quick Fix 1: services/pipeline.py (lines 305-310)
existing = session.query(db.StructuredRecord).filter_by(
    project_id=project_id, doc=filename, page=i + 1, entity=row_key, field=col
).first()
if existing:
    existing.value = val
    existing.raw_value = raw_val
    existing.confidence = "HIGH" if status == "SUCCESS" else "AMBIGUOUS"
    existing.status = status
    continue

# Quick Fix 2: main.py (_units_from_meta, lines 1943-1951)
inherited_tbl = (pg.get("header_inherited_tables") or "").strip()
if inherited_tbl:
    units.append(("[STRUCTURED TABLE]\n" + inherited_tbl, pg.get("page")))

# Quick Fix 3: intelligence/document_memory.py (line 571)
def is_allowlisted(filename: str) -> bool:
    from config import get_settings
    s = get_settings()
    if getattr(s, "document_memory_allow_all", True):
        return True
    return filename in (s.document_memory_doc_allowlist or [])
```

Audit Conclusion & Verification: Implementing these three Phase 1 quick fixes immediately enables full value extraction for any newly uploaded document while preserving 100% backward compatibility with all 267 existing unit and regression tests.